Vulnerability Report

Archivo: CustomerController.cs

Code Analyzed:

```
?using Castle.Core.Resource;
using FermaOrders.API.Controllers.Response;
using FermaOrders.Application.Dto.Components.Customer;
using FermaOrders.Application.Interface.Components;
using FermaOrders.Application.Service.Components;
using FermaOrders.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Data;
using System.Net;

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class CustomerController : ControllerBase
    {
        private readonly ICustomerService _customerService;

        public CustomerController(ICustomerService customerService)
        {
            _customerService = customerService;
        }

        // Crear un asiento
        [Authorize(Roles = "Admin")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateCustomerAsync([FromBody] CustomerCreateDto
customerDto)
        {
            var response = new RespuestaAPI();
            try
            {
                if (customerDto == null)
                {
                    return BadRequest("Invalid customer data.");
                }

                var customer = await _customerService.CreateCustomerAsync(customerDto);
                response.Result = customer;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        // Obtener un asiento por ID
        [Authorize(Roles = "Admin")]
        [HttpGet("{id}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetCustomerByIdAsync(int id)
        {
            var response = new RespuestaAPI();

            try
            {
```

```
                var customer = await _customerService.GetCustomerIdAsync(id);
                if (customer == null)
                {
                    return BadRequest("Invalid customer data.");
                }

                response.Result = customer;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }


        }

        [Authorize(Roles = "Admin")]
        [HttpPut("{id}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> UpdateCustomerAsync(int id, [FromBody]
CustomerUpdateDto customerDto)
        {
            var response = new RespuestaAPI();

            try
            {
                if (id != customerDto.Id)
                {
                    return BadRequest("ID mismatch.");
                }
                await _customerService.UpdateCustomerDto(customerDto);

                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                response.Result = new { message = "Customer updated successfully." };
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        // Eliminar un asiento
        [Authorize(Roles = "Admin")]
        [HttpDelete("{id}")]
        [ProducesResponseType(StatusCodes.Status204NoContent)]
        [ProducesResponseType(StatusCodes.Status404NotFound)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> DeleteCustomerAsync(int id)
        {
            var response = new RespuestaAPI();
            try
            {
                await _customerService.DeleteCustomerAsync(id);
                return NoContent();
            }
            catch (KeyNotFoundException ex)
            {
                response.StatusCode = HttpStatusCode.NotFound;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return NotFound(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
```

```
                return StatusCode(500, response);
            }
        }


        [Authorize(Roles = "Admin")]
        [HttpGet("customer")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetCustomerList()
        {
            var response = new RespuestaAPI();

            try
            {
                var customers = await _customerService.ListCustomer();

                response.Result = customers;
                response.IsSuccess = true;
                response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.OK;

                return Ok(response);
            }
            catch (Exception ex)
            {
                response.IsSuccess = false;
                response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError;
                response.ErrorMessages.Add($"Error: {ex.Message}");

                return StatusCode(500, response);
            }

            // Redundante, pero garantiza que todas las rutas retornan algo
            // No debería alcanzarse nunca
            return StatusCode(500, new RespuestaAPI
            {
                IsSuccess = false,
                StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError,
                ErrorMessages = new List<string> { "Ocurrió un error inesperado." }
            });
        }
    }
}
```

Analysis: ```html
Security Vulnerabilities and Code Quality Analysis

Identified Vulnerabilities

Vulnerability: Insecure Direct Object Reference (IDOR)

Approximate Line: All methods that take an `id` parameter (GetCustomerByIdAsync, UpdateCustomerAsync, DeleteCustomerAsync).
Description: The code relies on the `Admin` role for authorization, which verifies the user's role but it doesn't ensure the user has the right to access specific Customer's data. If an attacker guesses or obtains another Customer's ID, they can potentially access, modify, or delete that Customer's information even when they should not be able to. This is a problem, because an Admin can access all customer's data.
Mitigation: Implement authorization checks that verify the user has permission to access, modify, or delete the specific customer based on the provided `id`. This could involve checking ownership or specific permissions related to the customer being accessed. For instance, a "Manager" role might be able to access all customers within their "branch," while a regular "Employee" role can only access customers assigned to them. The data model should contain information on the relationship of customers to other entitities that can be used to apply the appropriate checks.

Vulnerability: Lack of Input Validation/Sanitization

Approximate Line: Throughout the controller, especially in `CreateCustomerAsync` and `UpdateCustomerAsync`.
Description: The code appears to perform minimal input validation beyond a null check. The controller should validate the values of the fields within `CustomerCreateDto` and `CustomerUpdateDto` to ensure they conform to the expected format and range (e.g., string length, valid email, phone number). This prevents invalid data from being stored in the database and potentially causing application errors or security vulnerabilities (e.g., SQL injection if the data is used in raw SQL queries, though this is less likely with ORMs).
Mitigation: Use data annotations within the DTOs and leverage model validation in the controller using `ModelState.IsValid`. Implement custom validation logic as needed for more complex validation rules. Sanitize potentially dangerous input, especially if the data is later displayed in a web page.
Code Quality Improvements: Using data annotations directly in the DTOs is preferable to custom validation logic in the controller, since the controller's job is not to validate incoming requests, but rather to process valid data according to some business logic.

Vulnerability: Inconsistent Error Handling

Approximate Line: Throughout the controller actions.
Description: The error handling in the controller repeats the same try-catch block in most methods which creates code duplication. The error handling is also inconsistent in that some methods return `NotFound` while others return `BadRequest` based on similar failure criteria.
Mitigation: Implement a centralized exception handling mechanism, such as a global exception filter or middleware. This reduces code duplication and ensures consistent error responses. Categorize exceptions and map them to appropriate HTTP status codes. Use consistent error messages for similar scenarios.
Code Quality Improvements: Reduce duplication by implementing custom exception filters or middleware to handle responses uniformly. Improves readability and maintainability.

Code Quality Metrics and Improvements

Identified Metrics

Metric: Duplication

Description: The `try-catch` blocks and `RespuestaAPI` object creation are highly duplicated across all controller actions. The redundant status code returns in `GetCustomerList` contributes as well.
Improvement: Implement a centralized exception handling mechanism (as described above) and a common method for creating `RespuestaAPI` objects. The last two lines of `GetCustomerList` can be removed since execution should never reach that point.

Metric: Readability

Description: While the code is generally readable, the repetitive `try-catch` blocks and verbose response creation can reduce readability.
Improvement: Centralizing exception handling and response creation significantly improves readability by reducing visual clutter and making the core logic of each action more apparent. Standardizing the naming conventions is a good practice for improving readability in general, which appears to be the case in the existing code.

Metric: Coupling

Description: The controller is tightly coupled to the `RespuestaAPI` object.
Improvement: Consider using a more generic IActionResult result, such as `Ok(object)` or `BadRequest(string)`, or implement a custom result filter that transforms the result of the service layer into the desired HTTP response. The exception handling middleware would catch exceptions and generate the appropriate error responses.

Proposed Solution

Summary

1. **Implement fine-grained authorization:** Instead of relying solely on role-based authorization, check if the user has permission to access the specific customer data being requested. Use data from the Customer, and other entities (such as Branch) to ensure an Admin has the proper authorization to access the data. 2. **Implement robust input validation:** Use data annotations in the DTOs and `ModelState.IsValid` in the controller. 3. **Centralize exception handling:** Use a global exception filter or middleware to handle exceptions consistently. 4. **Reduce code duplication:** Create helper methods for common tasks, such as creating `RespuestaAPI` objects. 5. **Decouple from `RespuestaAPI`:** Consider alternative ways to format the HTTP response.

```
```